----------Mandatory Information to fill----------

Group ID: 153

Group Members Name with Student ID:

1.Ashvini Satish Bhosale (Student ID: 2024aa05732)-Contrubition -100%

2.Rajeev Kumar Jha(Student ID: 2024aa05194)-Contrubition -100%

3.Dhruvi Nileshbhai Makati(Student ID: 2024aa05575)-Contrubition -100%

4.S Naresh Kumar(Student ID: 2024ab05178)-Contrubition -100%

----------Write your remarks (if any) that you want should get consider at the time of evaluation----------

Remarks: ##Add here

Autonomous Drone Battery Management for Urban Surveillance using DQN and DDQN - 7 Marks

Import Statements

In [23]:
```
# 0.1.1 Import Statements
# ==== Standard Library ====
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"  # or "3" for silence
import tensorflow as tf
tf.get_logger().setLevel("ERROR")
import os
# Suppress *all* TF logs, including device placement
```

```
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"      # 0=all, 1=INFO, 2=WARNING, 3=ERROR

# Also disable op placement logging
os.environ["TF_CPP_MIN_VLOG_LEVEL"] = "0"     # disables verbose placement logs

import tensorflow as tf
tf.get_logger().setLevel("ERROR")

# Optional (for TF v2.x with compat API)
tf.compat.v1.logging.set_verbosity(tf.compat.v1.logging.ERROR)


import math
import random
import time
from collections import deque
from dataclasses import dataclass
from pathlib import Path
from typing import List, Tuple, Optional, Deque, Dict

# ==== Third-Party ====
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import matplotlib.pyplot as plt
from tqdm.auto import tqdm

# ==== Device Selection (CUDA / Apple MPS / CPU) ====
# Automatically detect and set the device for PyTorch operations
device = torch.device(
    "cuda" if torch.cuda.is_available() else ("mps" if hasattr(torch.backends, "mps
)

# ==== Reproducibility Helper ====
# Function to set random seeds for reproducibility across different libraries
def set_seed(seed: int = 42) -> None:
    """
    Sets the random seed for Python's built-in random, NumPy, and PyTorch
    to ensure reproducibility of results.

    Args:
        seed (int): The seed value to use. Defaults to 42.
    """
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42) # Set the seed at the beginning of the script
```

In [24]:

```
# Environment Parameters
GRID_SIZE = (10, 10)  # X, Y dimensions of the urban area grid [1]
BATTERY_CAPACITY = 100.0  # Maximum battery level in percentage [1]
```

```
INITIAL_BATTERY = 100.0 # Starting battery level at the beginning of an episode
BASE_MOVE_COST = 0.5  # Base battery percentage cost per grid step [1]
BASE_HOVER_COST = 0.2  # Base battery percentage cost per time step when hovering [
RECHARGE_RATE = 5.0  # Battery percentage recharged per time step at a charging sta
BATTERY_CRASH_PENALTY = -100.0 # Reward penalty for the drone running out of batter
TIME_PENALTY = -0.1  # Small negative reward for each time step to encourage effici
RECHARGE_BONUS = 1.0  # Small positive reward for successfully recharging at a stat

# POI Parameters (Points of Interest)
POI_SPAWN_CHANCE = 0.05  # Probability of a new POI spawning at each time step [1]
MAX_ACTIVE_POIS = 3  # Maximum number of POIs that can be active simultaneously [1]
POI_LIFESPAN_RANGE = (10, 30)  # Minimum and maximum timesteps a POI remains active
POI_VALUE_RANGE = (10, 50)  # Minimum and maximum surveillance score gained from vi

# Atmospheric Disturbance Parameters
DISTURBANCE_FACTOR = 0.5  # Multiplier that scales the effect of disturbance on bat
DISTURBANCE_CHANGE_PROB = 0.1  # Probability that the atmospheric disturbance level
DISTURBANCE_MAGNITUDE_CHANGE = 0.1  # Maximum amount the disturbance level can chan

# Agent Parameters (DQN/DDQN)
STATE_SIZE = 7  # Dimension of the observation space (input features for the agent'
ACTION_SIZE = 6  # Number of possible actions the drone can take (North, South, Eas

LEARNING_RATE = 0.001 # Learning rate for the optimizer [Placeholder for example va
DISCOUNT_FACTOR = 0.99  # Discount factor (gamma) for future rewards [3]
GAMMA = 0.99  # Discount factor (gamma) for future rewards (duplicate, keep for cla

REPLAY_BUFFER_SIZE = 100000 # Maximum number of experiences to store in the replay
MIN_REPLAY_SIZE = 1000  # Number of random transitions to pre-fill the replay buffe
BATCH_SIZE = 32 # Number of experiences sampled from the replay buffer for each lea

EXPLORATION_MAX = 1.0 # Initial value of epsilon for the epsilon-greedy exploration
EXPLORATION_MIN = 0.01 # Minimum value of epsilon [Placeholder for example value]
EXPLORATION_DECAY = 0.995 # Decay rate for epsilon after each learning step [3]

TARGET_UPDATE_FREQUENCY = 100 # Frequency (in total steps) at which the target netw

# Training Parameters
EPISODES = 10  # Total number of training episodes (each representing a simulated o
MAX_TIMESTEPS_PER_EPISODE = 200  # Maximum number of steps allowed in a single epis
```

In [25]:

```
# --- 2. Replay Buffer Class ---
# This class implements a replay buffer to store and sample experiences for trainin
# the DQN/DDQN agent, helping to break temporal correlations. [3]

@dataclass
class Experience:
    """
    A simple data class to hold a single experience tuple.
    """
    state: np.ndarray
    action: int
    reward: float
    next_state: np.ndarray
    done: bool
```

```python
class ReplayBuffer:
    def __init__(self, capacity: int):
        """
        Initializes the ReplayBuffer with a given maximum capacity.
        The buffer stores 'Experience' tuples (state, action, reward, next_state, d

        Args:
            capacity (int): The maximum number of experiences the buffer can hold.
        """
        self.buffer = deque(maxlen=capacity) # Use deque with maxlen for efficient

    def add(self, state: np.ndarray, action: int, reward: float, next_state: np.nda
        """
        Adds a new experience to the buffer.
        If the buffer is full (reached maxlen), the oldest experience is automatica

        Args:
            state (np.ndarray): The state before the action was taken.
            action (int): The action taken.
            reward (float): The reward received after taking the action.
            next_state (np.ndarray): The state after taking the action.
            done (bool): A boolean indicating if the episode terminated after this
        """
        experience = Experience(state, action, reward, next_state, done)
        self.buffer.append(experience) # Append the new experience to the right sid

    def sample(self, batch_size: int) -> Tuple[np.ndarray, np.ndarray, np.ndarray,
        """
        Randomly samples a batch of experiences from the buffer.

        Args:
            batch_size (int): The number of experiences to sample.

        Returns:
            A tuple of numpy arrays: (states, actions, rewards, next_states, dones)
            Each array contains the respective components from the sampled batch.

        Raises:
            ValueError: If the buffer contains fewer experiences than the requested
        """
        if len(self.buffer) < batch_size:
            raise ValueError("Not enough experiences in the buffer to sample the re

        batch = random.sample(self.buffer, batch_size) # Randomly sample 'batch_siz

        # Separate the components into individual numpy arrays for efficient proces
        states = np.array([e.state for e in batch], dtype=np.float32)
        actions = np.array([e.action for e in batch], dtype=np.int64)
        rewards = np.array([e.reward for e in batch], dtype=np.float32)
        next_states = np.array([e.next_state for e in batch], dtype=np.float32)
        # Using float for consistency with PyTorch boolean handling (True=1.0, Fals
        dones = np.array([e.done for e in batch], dtype=np.float32)

        return states, actions, rewards, next_states, dones

    def __len__(self) -> int:
```

```
        """
        Returns the current number of experiences stored in the buffer.
        """
        return len(self.buffer)

# Instantiate the replay buffer (This line should be placed where the buffer is fir
# replay_buffer = ReplayBuffer(REPLAY_BUFFER_SIZE) # This line would typically be p
```

--- 3. Custom Environment: DroneSurveillanceEnv --- - 2 Marks

In [26]:

```
import math
import random
import numpy as np
from typing import List, Tuple, Dict, Any, Optional

class DroneSurveillanceEnv:
    """
    A custom OpenAI Gym-like environment for simulating an autonomous drone
    performing surveillance tasks in an urban grid environment with
    dynamic battery levels, points of interest (POIs), and atmospheric disturbances
    """
    def __init__(self, grid_size: Tuple[int, int], battery_capacity: float, initial
        """
        Initializes the Drone Surveillance Environment.

        Args:
            grid_size (Tuple[int, int]): Dimensions of the 2D urban area grid (X, Y
            battery_capacity (float): Maximum battery level (e.g., 100.0%).
            initial_battery (float): Starting battery level for the drone at the st
        """
        # Environment Parameters (defined in constants)
        self.grid_size = grid_size
        self.battery_capacity = battery_capacity
        self.initial_battery = initial_battery
        self.base_move_cost = BASE_MOVE_COST
        self.base_hover_cost = BASE_HOVER_COST
        self.recharge_rate = RECHARGE_RATE
        self.battery_crash_penalty = BATTERY_CRASH_PENALTY
        self.time_penalty = TIME_PENALTY
        self.recharge_bonus = RECHARGE_BONUS

        # POI Parameters (defined in constants)
        self.poi_spawn_chance = POI_SPAWN_CHANCE
        self.max_active_pois = MAX_ACTIVE_POIS
        self.poi_lifespan_range = POI_LIFESPAN_RANGE
        self.poi_value_range = POI_VALUE_RANGE

        # Atmospheric Disturbance Parameters (defined in constants)
        self.disturbance_factor = DISTURBANCE_FACTOR
        self.disturbance_change_prob = DISTURBANCE_CHANGE_PROB
        self.disturbance_magnitude_change = DISTURBANCE_MAGNITUDE_CHANGE

        # Drone State Initialization [2, 3]
        self.drone_pos = np.array([0, 0], dtype=int)  # [X, Y] - Drone's current po
```

```
        self.battery_level = self.initial_battery # Drone's current battery level

        # Environmental Dynamics Initialization [3]
        self.atmospheric_disturbance = 0.0  # Current level of wind/turbulence (0.0
        # List to store active POIs. Each POI is a dictionary.
        self.active_pois: List[Dict[str, Any]] = [] # List of active POIs: {'pos':

        # Charging Stations (fixed locations) [2, 4]
        # Example fixed locations. Can be defined based on scenario needs.
        self.charging_stations = [
            np.array([3, 2]),
            np.array([self.grid_size[0] - 1, 0]),
            np.array([0, self.grid_size[1] - 1]),
            np.array([self.grid_size[0] - 1, self.grid_size[1] - 1]),
            np.array([self.grid_size[0] // 2, self.grid_size[1] // 2]) # Center
        ]

        # Action mapping: 0:N, 1:S, 2:E, 3:W, 4:Hover, 5:Recharge [4, 6]
        self.action_space = list(range(ACTION_SIZE)) # Total number of possible act
        # Dictionary mapping action integers to movement vectors (dx, dy)
        self.action_map = {
            0: (0, 1),   # North (dy = +1)
            1: (0, -1),  # South (dy = -1)
            2: (1, 0),   # East (dx = +1)
            3: (-1, 0),  # West (dx = -1)
            4: (0, 0),   # Hover (no movement)
            5: (0, 0)    # Recharge (no movement, must be at station)
        }

        # Episode tracking variables
        self.current_timestep = 0 # Counter for the current step within an episode
        self.total_surveillance_score = 0.0 # Cumulative surveillance score collect

    def reset(self) -> np.ndarray:
        """
        Resets the environment to its initial state for a new episode. [1]
        This includes resetting drone position and battery, clearing POIs,
        and randomizing atmospheric disturbance.

        Returns:
            np.ndarray: The initial observation of the environment.
        """
        self.drone_pos = np.array([0, 0], dtype=int)  # Place drone at a starting l
        self.battery_level = self.initial_battery  # Reset battery to full
        self.active_pois = []  # Clear active POIs
        self.atmospheric_disturbance = random.uniform(0.0, 1.0)  # Set initial rand
        self.current_timestep = 0 # Reset timestep counter
        self.total_surveillance_score = 0.0 # Reset surveillance score
        return self._get_obs() # Return the initial observation

    def _get_obs(self) -> np.ndarray:
        """
        Converts the internal environment state into the observation vector for the
        This observation includes drone position, battery, disturbance, and informa
        about the nearest active POI.
```

```
        The state size is 7: [drone_x, drone_y, battery_level, disturbance,
                              nearest_poi_dist, nearest_poi_value, nearest_poi_life

        Returns:
            np.ndarray: The observation vector representing the current state.
        """
        # Initialize nearest POI information with default values
        # Max possible Manhattan distance in the grid
        nearest_poi_dist = self.grid_size[0] + self.grid_size[1] - 2 # Adjusted for
        nearest_poi_value = 0.0
        nearest_poi_lifespan = 0

        if self.active_pois:
            # Find the nearest POI using Manhattan distance
            min_dist = float('inf')
            closest_poi = None
            for poi in self.active_pois:
                poi_pos = poi['pos']
                # Calculate Manhattan distance between drone and POI
                dist = abs(self.drone_pos[0] - poi_pos[0]) + abs(self.drone_pos[1]
                if dist < min_dist:
                    min_dist = dist
                    closest_poi = poi

            if closest_poi:
                nearest_poi_dist = min_dist
                nearest_poi_value = closest_poi['value']
                nearest_poi_lifespan = closest_poi['lifespan']

        # Normalize or scale values for better neural network performance
        # Coordinates are normalized by grid size
        # Battery is normalized by battery capacity
        # Disturbance is already 0-1
        # POI distance is normalized by maximum possible Manhattan distance
        # POI value is normalized by maximum possible POI value
        # POI lifespan is normalized by maximum possible POI lifespan
        state = np.array([
            self.drone_pos[0] / (self.grid_size[0] - 1),  # Normalized X-coordinate
            self.drone_pos[1] / (self.grid_size[1] - 1),  # Normalized Y-coordinate
            self.battery_level / self.battery_capacity,   # Normalized Battery Perc
            self.atmospheric_disturbance,                 # Current Atmospheric Dis
            nearest_poi_dist / (self.grid_size[0] + self.grid_size[1] - 2), # Norma
            nearest_poi_value / max(self.poi_value_range), # Normalized POI value
            nearest_poi_lifespan / max(self.poi_lifespan_range) # Normalized POI li
        ], dtype=np.float32)

        return state

    def _spawn_poi(self):
        """
        Attempts to spawn a new POI randomly on an empty cell with a certain probab
        An empty cell is one not occupied by the drone, a charging station, or anot
        Spawning is limited by MAX_ACTIVE_POIS.
        """
        # Check if conditions for spawning a new POI are met
        if random.random() < self.poi_spawn_chance and len(self.active_pois) < self
```

```
            # Find all currently unoccupied cells in the grid
            empty_cells = []
            for r in range(self.grid_size[0]):
                for c in range(self.grid_size[1]):
                    pos = np.array([r, c])
                    is_occupied = False
                    # Check if occupied by drone
                    if np.array_equal(pos, self.drone_pos):
                        is_occupied = True
                    # Check if occupied by charging station
                    if any(np.array_equal(pos, cs_pos) for cs_pos in self.charging_
                        is_occupied = True
                    # Check if occupied by existing POI
                    if any(np.array_equal(pos, poi['pos']) for poi in self.active_p
                        is_occupied = True

                    if not is_occupied:
                        empty_cells.append(pos)

            # If empty cells exist, select one randomly and create a new POI
            if empty_cells:
                new_poi_pos = random.choice(empty_cells)
                new_poi_value = random.uniform(*self.poi_value_range)
                new_poi_lifespan = random.randint(*self.poi_lifespan_range)
                self.active_pois.append({
                    'pos': new_poi_pos,
                    'value': new_poi_value,
                    'lifespan': new_poi_lifespan
                })

    def _update_pois(self, drone_at_poi_pos: Optional[Tuple[int, int]]) -> float:
        """
        Updates the lifespans of active POIs and handles the collection of rewards
        if the drone is currently at a POI's position. [8, 9]

        Args:
            drone_at_poi_pos (Optional[Tuple[int, int]]): The position (x, y) of a
                                                          if the drone is currently

        Returns:
            float: The total reward collected from visiting POIs in this timestep.
        """
        poi_reward = 0.0
        # Create a new list for POIs that remain active after updates
        remaining_pois = []
        for poi in self.active_pois:
            # Check if the drone is currently at this POI's position
            if drone_at_poi_pos is not None and np.array_equal(poi['pos'], drone_at
                poi_reward += poi['value']  # Collect POI value as reward
                # This POI is collected and disappears, so it's not added to remain
            else:
                poi['lifespan'] -= 1  # Decrease lifespan by one timestep
                if poi['lifespan'] > 0:
                    remaining_pois.append(poi) # Keep the POI if its lifespan is st
        self.active_pois = remaining_pois # Update the list of active POIs
        return poi_reward
```

```
    def _update_atmospheric_disturbance(self):
        """
        Updates the atmospheric disturbance randomly each time step based on a
        probability and magnitude change. [9]
        The disturbance level is kept within the range [0.0, 1.0].
        """
        if random.random() < self.disturbance_change_prob: # Check if disturbance c
            # Calculate a random change within the defined magnitude
            change = random.uniform(-self.disturbance_magnitude_change, self.distur
            # Apply the change and clip the disturbance level to the valid range [0
            self.atmospheric_disturbance = np.clip(self.atmospheric_disturbance + c

    def step(self, action: int) -> Tuple[np.ndarray, float, bool, Dict[str, Any]]:
        """
        Executes one step in the environment given an action taken by the drone. [1
        This involves updating drone position, battery, atmospheric disturbance,
        POI states, calculating reward, and checking for episode termination.

        Args:
            action (int): The action taken by the drone (0-5, mapping defined in se

        Returns:
            Tuple[np.ndarray, float, bool, Dict[str, Any]]:
                - next_state (np.ndarray): The observed state of the environment af
                - reward (float): The reward received in this timestep.
                - done (bool): True if the episode has ended, False otherwise.
                - info (Dict[str, Any]): Additional information (e.g., battery leve
        """
        reward = 0.0 # Initialize reward for the current timestep
        done = False # Initialize episode termination flag
        drone_at_poi_pos: Optional[Tuple[int, int]] = None # To track if a POI is c

        # --- Update Atmospheric Disturbance --- [9]
        self._update_atmospheric_disturbance()

        # Calculate energy cost modifier based on the current disturbance level
        energy_modifier = 1.0 + (self.atmospheric_disturbance * self.disturbance_fa

        # --- Battery Consumption / Recharge based on action --- [6, 8]
        if action == 5:  # Recharge action
            # Check if the drone is currently at any charging station location
            if any(np.array_equal(self.drone_pos, cs_pos) for cs_pos in self.chargi
                # If at a charging station, recharge the battery
                self.battery_level = min(self.battery_capacity, self.battery_level
                reward += self.recharge_bonus # Give a small bonus for successful r
            else:
                # If tried to recharge not at a station, penalize by consuming hove
                self.battery_level -= (self.base_hover_cost * energy_modifier)
        elif action == 4: # Hover action
            # Consume battery for hovering, modified by disturbance
            self.battery_level -= (self.base_hover_cost * energy_modifier)
        else: # Movement actions (North, South, East, West)
            # Get the movement vector (dx, dy) for the chosen action
            dx, dy = self.action_map[action]
            new_x = self.drone_pos[0] + dx
```

```
            new_y = self.drone_pos[1] + dy

            # Check if the new position is within the grid boundaries [10]
            if 0 <= new_x < self.grid_size[0] and 0 <= new_y < self.grid_size[1]:
                self.drone_pos = np.array([new_x, new_y], dtype=int) # Update drone
            # Apply movement cost even if hitting boundary (simulates attempt to mo
            self.battery_level -= (self.base_move_cost * energy_modifier)

        # Clamp battery level to ensure it stays within [0, battery_capacity]
        self.battery_level = np.clip(self.battery_level, 0.0, self.battery_capacity

        # --- Check for Crash (Battery Depletion) --- [8, 11]
        # If battery is 0 or less and the drone is NOT at a charging station, it cr
        if self.battery_level <= 0 and not any(np.array_equal(self.drone_pos, cs_po
            reward += self.battery_crash_penalty # Apply crash penalty
            done = True # Episode terminates on crash

        # --- Update POIs and Collect Rewards --- [8, 9]
        # Check if the drone is currently at the location of any active POI
        drone_on_poi = False
        for poi in self.active_pois:
            if np.array_equal(self.drone_pos, poi['pos']):
                drone_at_poi_pos = tuple(poi['pos']) # Store position of collected
                drone_on_poi = True
                break # Assuming drone can only collect one POI per step

        # Update POI lifespans and get reward from collected POIs
        poi_collected_reward = self._update_pois(drone_at_poi_pos)
        reward += poi_collected_reward # Add collected POI reward to total reward
        self.total_surveillance_score += poi_collected_reward # Accumulate surveill

        # --- Spawn new POIs randomly --- [8]
        self._spawn_poi()

        # --- Add general time penalty --- [11]
        reward += self.time_penalty # Apply a small penalty for each timestep
        self.current_timestep += 1 # Increment the timestep counter

        # Episode termination conditions [1, 11]
        # The episode also ends if the maximum number of timesteps is reached.
        if self.current_timestep >= MAX_TIMESTEPS_PER_EPISODE:
            done = True

        # Get the observation for the next state
        next_state = self._get_obs()
        # Prepare additional information to return (optional, for analysis)
        info = {
            "battery_level": self.battery_level,
            "drone_pos": self.drone_pos,
            "atmospheric_disturbance": self.atmospheric_disturbance,
            "active_pois_count": len(self.active_pois),
            "surveillance_score": self.total_surveillance_score
        }

        return next_state, reward, done, info
```

```
    def render(self):
        """
        Prints a textual representation of the grid environment to the console. [10
        Shows the locations of the drone (D), charging stations (C),
        active points of interest (P), and empty cells (.).
        Also displays key state information like drone position, battery, and distu
        """
        # Create a grid representation initialized with empty cells
        grid = [['.' for _ in range(self.grid_size[0])] for _ in range(self.grid_si

        # Mark Charging Stations on the grid
        for cs_pos in self.charging_stations:
            # Ensure charging station position is within bounds before marking
            if 0 <= cs_pos[0] < self.grid_size[0] and 0 <= cs_pos[1] < self.grid_si
                grid[cs_pos[1]][cs_pos[0]] = 'C' # Mark with 'C'

        # Mark Active POIs on the grid
        for poi in self.active_pois:
            poi_pos = poi['pos']
            # Ensure POI position is within bounds before marking
            if 0 <= poi_pos[0] < self.grid_size[0] and 0 <= poi_pos[1] < self.grid_
                grid[poi_pos[1]][poi_pos[0]] = 'P' # Mark with 'P'

        # Mark Drone's position on the grid (overwrites other markers if on same sp
        if 0 <= self.drone_pos[0] < self.grid_size[0] and 0 <= self.drone_pos[1] <
            grid[self.drone_pos[1]][self.drone_pos[0]] = 'D' # Mark with 'D'

        # Print the grid with a border
        print("-" * (self.grid_size[0] * 2 + 3))
        for row in grid:
            print("| " + " ".join(row) + " |")
        print("-" * (self.grid_size[0] * 2 + 3))
        # Print additional state information
        print(f"Drone Pos: {self.drone_pos}, Battery: {self.battery_level:.1f}%, Di
        print(f"Active POIs: {len(self.active_pois)}, Score: {self.total_surveillan

# Example instantiation (this line would typically be in the main execution block l
# env = DroneSurveillanceEnv(GRID_SIZE, BATTERY_CAPACITY, INITIAL_BATTERY)
```

--- 4. DQNAgent Class --- 1 Mark

In [27]:

```
# Imports for the DQNAgent class [1]
import numpy as np
import random
from collections import deque
import tensorflow as tf
from tensorflow.keras.models import Sequential, clone_model
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

class DQNAgent:
    """
    Implements a Deep Q-Network (DQN) or Double Deep Q-Network (DDQN) agent
    for reinforcement learning.
    """
```

```
    def __init__(self, state_size: int, action_size: int, replay_buffer: ReplayBuff
        """
        Initializes the DQNAgent.

        Args:
            state_size (int): The size of the observation space.
            action_size (int): The number of possible actions.
            replay_buffer (ReplayBuffer): An instance of the ReplayBuffer class.
            use_ddqn (bool): If True, use Double DQN; otherwise, use standard DQN.
        """
        self.state_size = state_size
        self.action_size = action_size
        self.use_ddqn = use_ddqn

        # Replay buffer for experience storage and sampling
        self.replay_buffer = replay_buffer

        # Hyperparameters (directly from constants section)
        self.gamma = GAMMA # Discount factor for future rewards
        self.epsilon = EXPLORATION_MAX # Initial value for epsilon-greedy explorati
        self.initial_epsilon = EXPLORATION_MAX  # Save initial value for resetting
        self.epsilon_min = EXPLORATION_MIN # Minimum value for epsilon
        self.epsilon_decay = EXPLORATION_DECAY # Rate of decay for epsilon
        self.learning_rate = LEARNING_RATE # Learning rate for the optimizer
        self.batch_size = BATCH_SIZE # Size of the batch sampled from the replay bu

        # Main and target networks [2]
        # The main model learns the Q-values, and the target model provides stable
        self.model = self._build_model() # The main Q-network
        self.target_model = clone_model(self.model) # Creates a copy of the main mo
        # Copy weights from the main model to the target model initially
        self.target_model.set_weights(self.model.get_weights()) # Copies weights fr

    def reset_epsilon(self):
        """
        Resets epsilon to its initial maximum value. Useful for starting new traini
        """
        self.epsilon = self.initial_epsilon

    def _build_model(self):
        """
        Builds a simple fully-connected Q-network using TensorFlow/Keras. [3]

        The network architecture consists of:
        - An input layer that matches the state size.
        - Two hidden layers, each with 64 units and ReLU activation.
        - An output layer with 'action_size' units and linear activation,
          representing the predicted Q-value for each action. [3]

        The model is compiled using Mean Squared Error (MSE) as the loss function
        and the Adam optimizer.
        """
        model = Sequential()
        # Input layer and first hidden layer with 64 units and ReLU activation [3]
        model.add(Dense(64, input_dim=self.state_size, activation='relu'))
        # Second hidden layer with 64 units and ReLU activation [3]
```

```
        model.add(Dense(64, activation='relu'))
        # Output layer with 'action_size' units and linear activation (Q-values for
        model.add(Dense(self.action_size, activation='linear'))
        # Compiles the model using Mean Squared Error (MSE) loss and Adam optimizer
        model.compile(loss='mse', optimizer=Adam(learning_rate=self.learning_rate))
        return model

    def remember(self, state: np.ndarray, action: int, reward: float, next_state: n
        """
        Stores an experience tuple (state, action, reward, next_state, done) in the
        This method is called after each step in the environment.

        Args:
            state (np.ndarray): The state before the action.
            action (int): The action taken.
            reward (float): The reward received.
            next_state (np.ndarray): The state after the action.
            done (bool): Whether the episode ended.
        """
        # Assumes 'replay_buffer' is an instantiated object of a Replay Buffer clas
        self.replay_buffer.add(state, action, reward, next_state, done)

    def choose_action(self, state: np.ndarray) -> int:
        """
        Selects an action using an epsilon-greedy policy. [4]

        Args:
            state (np.ndarray): The current state observation.

        Returns:
            int: The chosen action (an integer corresponding to an action in the en
        """
        # With probability 'epsilon', choose a random action for exploration. [4]
        if np.random.rand() < self.epsilon:
            return random.randrange(self.action_size)
        # Otherwise, choose the action with the highest predicted Q-value from the
        # Predict Q-values for the current state using the main model [4]
        # np.expand_dims is used to add a batch dimension to the state
        q_values = self.model.predict(np.expand_dims(state, axis=0), verbose=0)
        return np.argmax(q_values[0]) # Return the index of the action with the hig

    def learn(self):
        """
        Updates the Q-values of the main network by sampling a batch from the repla
        and performing a gradient descent step. [4]

        This method implements the Q-value update rule for both standard DQN and Do
        It then trains the main model on this batch using the calculated target Q-v
        """
        # Checks if there are enough samples in the replay buffer to form a batch r
        if len(self.replay_buffer) < self.batch_size:
            return # Not enough samples, so skip the learning step.

        # Samples a batch of experiences from the replay buffer.
        states, actions, rewards, next_states, dones = self.replay_buffer.sample(se
```

```
        # Predict Q-values for next states based on the chosen DQN variant. [5]
        if self.use_ddqn:
            # Double DQN (DDQN):
            # 1. Use the main model to select the best action in the next state.
            # 2. Use the target model to evaluate the Q-value for that selected act
            # This decouples action selection from value evaluation, reducing overe
            next_actions = np.argmax(self.model.predict(next_states, verbose=0), ax
            q_next_target = self.target_model.predict(next_states, verbose=0) # Q-v
            # Calculate target Q-values: R + gamma * Q_target(S', A*), where A* is
            target_q = rewards + self.gamma * q_next_target[np.arange(self.batch_si
        else:
            # Standard DQN:
            # Use the target network to both select and evaluate the action in the
            # The maximum Q-value from the target network is used to calculate the
            q_next_target = self.target_model.predict(next_states, verbose=0) # Q-v
            # Calculate target Q-values: R + gamma * max(Q_target(S', A'))
            target_q = rewards + self.gamma * np.max(q_next_target, axis=1) * (1 -

        # Get the current Q-values predicted by the main model for the 'states' in
        q_values = self.model.predict(states, verbose=0)
        # Update the Q-value for the specific action taken in the batch with the ca
        # This is the target Q-value used for training the main network.
        q_values[np.arange(self.batch_size), actions] = target_q

        # Train the main model on the states and the calculated target Q-values. [6
        # This performs one gradient descent step to update the main network's weig
        self.model.fit(states, q_values, epochs=1, verbose=0)

        # Decay epsilon to reduce exploration over time, encouraging exploitation o
        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay

    def update_target_network(self):
        """
        Copies the weights from the main Q-network to the target Q-network. [6]

        This process is performed periodically (controlled by TARGET_UPDATE_FREQUEN
        to provide stable targets for the Q-value updates, which helps mitigate
        instability that could arise from using a single network for both predictio
        and target generation. [6]
        """
        self.target_model.set_weights(self.model.get_weights())
```

--- 5. Main Training Loop --- 1 Mark

In [28]:

```
# ----------------------------
# Section: Training Loop Functions
# ----------------------------
import numpy as np
import random
import tensorflow as tf

# Configure TensorFlow to allow soft device placement and suppress logging device p
tf.config.set_soft_device_placement(True)
tf.debugging.set_log_device_placement(False)
```

```
def prefill_buffer(env: DroneSurveillanceEnv, steps: int = MIN_REPLAY_SIZE):
    """
    Pre-fills the global replay buffer with experiences collected by taking random
    in the environment. This is done before training starts to ensure the buffer ha
    a minimum number of diverse experiences. [3]

    Args:
        env (DroneSurveillanceEnv): The environment instance.
        steps (int): The number of steps to run to pre-fill the buffer. Defaults to
    """
    state = env.reset() # Reset the environment to get the initial state
    print(f"Prefilling replay buffer with {steps} random steps...")
    for _ in range(steps):
        # Choose a random action from the environment's action space
        action = random.choice(env.action_space)
        # Take a step in the environment with the chosen action
        next_state, reward, done, _ = env.step(action)
        # Add the collected experience (state, action, reward, next_state, done) to
        replay_buffer.add(state, action, reward, next_state, done)  # Assumes globa
        # Update the current state, resetting the environment if the episode is don
        state = env.reset() if done else next_state
    print("Replay buffer pre-filling complete.")


def train_agent(agent: DQNAgent, env: DroneSurveillanceEnv, episodes: int = EPISODE
                steps_per_episode: int = MAX_TIMESTEPS_PER_EPISODE,
                agent_name: str = "Agent", render: bool = False, save_initial_weigh
    """
    Trains a DQN/DDQN agent in the DroneSurveillanceEnv for a specified number of e
    Includes exploration decay, periodic target network updates, and logging.

    Args:
        agent (DQNAgent): The agent instance to train (DQN or DDQN).
        env (DroneSurveillanceEnv): The environment instance.
        episodes (int): The total number of episodes to train for. Defaults to EPIS
        steps_per_episode (int): The maximum number of steps per episode. Defaults
        agent_name (str): A name for the agent (used for logging). Defaults to "Age
        render (bool): If True, render the environment visually during training. De
        save_initial_weights (bool): If True, save the agent's initial model weight

    Returns:
        Dict[str, List[float] or Any]: A dictionary containing training results:
            - "episode_rewards": List of total rewards for each episode.
            - "epsilon_values": List of epsilon values at the end of each episode.
            - "episode_lengths": List of the number of steps taken in each episode.
            - "initial_weights": (Optional) The initial weights of the agent's mode
    """
    episode_rewards = [] # List to store total rewards for each episode
    epsilon_values = [] # List to store epsilon values at the end of each episode
    episode_lengths = [] # List to store the number of steps taken in each episode

    # Save initial weights if requested
    initial_weights = agent.model.get_weights() if save_initial_weights else None

    # Pre-fill replay buffer before starting the main training loop
```

```
    if len(replay_buffer) < MIN_REPLAY_SIZE:
         prefill_buffer(env, steps=MIN_REPLAY_SIZE - len(replay_buffer))


    total_steps = 0 # Counter for total steps across all episodes
    for ep in range(episodes):
        print(f"[{agent_name}] Episode {ep+1}/{episodes} starting...") # Log episoa

        state = env.reset() # Reset the environment for a new episode
        total_reward = 0 # Initialize total reward for the current episode
        steps_this_ep = 0 # Initialize step counter for the current episode


        # Run the episode for a maximum number of steps
        for step in range(steps_per_episode):
            if render:
                env.render() # Render the environment if enabled

            # Agent chooses an action based on the current state and epsilon-greedy
            action = agent.choose_action(state)

            # Take a step in the environment with the chosen action
            next_state, reward, done, _ = env.step(action)

            # Store the experience in the replay buffer
            agent.remember(state, action, reward, next_state, done)

            # Perform a learning step (update the main network) if enough experienc
            agent.learn()

            # Update the current state and accumulate reward
            state = next_state
            total_reward += reward
            steps_this_ep += 1
            total_steps += 1 # Increment total steps

            # Update the target network periodically
            if total_steps % TARGET_UPDATE_FREQUENCY == 0:
                agent.update_target_network()

            # If the episode is done (e.g., crashed or reached max steps), break th
            if done:
                break

        # Store results for the completed episode
        episode_rewards.append(total_reward)
        epsilon_values.append(agent.epsilon)
        episode_lengths.append(steps_this_ep)

        # Log training progress periodically
        if (ep + 1) % 10 == 0: # Log every 10 episodes
            # Calculate average reward and steps over the last 10 episodes
            avg_last_10_reward = np.mean(episode_rewards[-10:])
            avg_last_10_steps = np.mean(episode_lengths[-10:])
            print(f"[{agent_name}] Episode {ep + 1}/{episodes} "
                  f"- Avg Reward (last 10): {avg_last_10_reward:.2f} "
```

```
                f"- Epsilon: {agent.epsilon:.3f} "
                f"- Avg Steps: {avg_last_10_steps:.1f}")

    # Prepare results dictionary to return
    results = {
        "episode_rewards": episode_rewards,
        "epsilon_values": epsilon_values,
        "episode_lengths": episode_lengths
    }

    # Add initial weights to results if requested
    if save_initial_weights:
        results["initial_weights"] = initial_weights

    return results

def moving_average(data: List[float], window: int = 20) -> np.ndarray:
    """
    Computes the moving average of a list of numerical data. [1]
    Used for smoothing learning curves to better visualize trends.

    Args:
        data (List[float]): The list of numerical data.
        window (int): The size of the moving average window. Defaults to 20.

    Returns:
        np.ndarray: A numpy array containing the smoothed moving average.
    """
    # Use np.convolve with a window of ones to compute the sum, then divide by wind
    # 'valid' mode returns output only where the convolution fully overlaps with th
    return np.convolve(data, np.ones(window) / window, mode='valid')
```

--- Main Execution Block ---

In [29]:

```
if __name__ == "__main__":
    # Ensure TensorFlow uses GPU if available
    # This was already added to the training loop cell, but can be re-verified here
    env = DroneSurveillanceEnv(GRID_SIZE, BATTERY_CAPACITY, INITIAL_BATTERY)
    replay_buffer = ReplayBuffer(REPLAY_BUFFER_SIZE) # Instantiate the replay buffe

    # --- Train DQN Agent ---
    print("Training DQN Agent...")
   # replay_buffer = ReplayBuffer(REPLAY_BUFFER_SIZE)

    dqn_agent = DQNAgent(STATE_SIZE, ACTION_SIZE, replay_buffer, use_ddqn=False)
    dqn_agent.reset_epsilon()
    dqn_results = train_agent(dqn_agent, env, episodes=EPISODES,
                              steps_per_episode=MAX_TIMESTEPS_PER_EPISODE,
                              agent_name="DQN", render=False)


    # --- Train Double DQN Agent ---
    print("\nTraining Double DQN Agent...")
    ddqn_agent = DQNAgent(STATE_SIZE, ACTION_SIZE, replay_buffer, use_ddqn=True)
    ddqn_agent.reset_epsilon()
```

```
    ddqn_results = train_agent(ddqn_agent, env, episodes=EPISODES,
                               steps_per_episode=MAX_TIMESTEPS_PER_EPISODE,
                               agent_name="DDQN", render=False)



    # --- Plotting Results ---
    print("\nPlotting results...")

    # Plot Episode Rewards
    plt.figure(figsize=(12, 6))
    #plt.plot(moving_average(dqn_results["episode_rewards"]), label="DQN (Smoothed)
    plt.plot(moving_average(dqn_results["episode_rewards"]), label="DQN (Smoothed)"
    plt.plot(moving_average(ddqn_results["episode_rewards"]), label="DDQN (Smoothed
    plt.xlabel("Episode")
    plt.ylabel("Total Reward (Moving Average)")
    plt.title("DQN vs. DDQN: Episode Rewards Over Training")
    plt.legend()
    plt.grid(True)
    plt.show()

    # Plot Epsilon Decay
    plt.figure(figsize=(12, 6))
    plt.plot(dqn_results["epsilon_values"], label="DQN Epsilon")
    plt.plot(ddqn_results["epsilon_values"], label="DDQN Epsilon")
    plt.xlabel("Episode")
    plt.ylabel("Epsilon")
    plt.title("Epsilon Decay Over Training")
    plt.legend()
    plt.grid(True)
    plt.show()

    # Plot Episode Lengths
    plt.figure(figsize=(12, 6))
    plt.plot(moving_average(dqn_results["episode_lengths"]), label="DQN (Smoothed)"
    plt.plot(moving_average(ddqn_results["episode_lengths"]), label="DDQN (Smoothed
    plt.xlabel("Episode")
    plt.ylabel("Episode Length (Moving Average)")
    plt.title("DQN vs. DDQN: Episode Lengths Over Training")
    plt.legend()
    plt.grid(True)
    plt.show()

    # --- Policy Analysis ---
    print("\nAnalyzing learned policy (for DDQN agent)...")

    # Evaluate the final policy of the DDQN agent by running a few episodes without
    ddqn_agent_eval = DQNAgent(STATE_SIZE, ACTION_SIZE, replay_buffer, use_ddqn=Tru
    ddqn_agent_eval.model.set_weights(ddqn_agent.model.get_weights()) # Copy traine
    ddqn_agent_eval.epsilon = 0.0 # No exploration during evaluation

    num_eval_episodes = 10
    eval_rewards = []
    eval_steps = []
    print(f"Running {num_eval_episodes} evaluation episodes with DDQN agent (epsilo
    for ep in range(num_eval_episodes):
```

```
        state = env.reset()
        total_reward = 0
        steps_this_ep = 0
        done = False
        while not done and steps_this_ep < MAX_TIMESTEPS_PER_EPISODE:
            action = ddqn_agent_eval.choose_action(state) # Use learned policy
            state, reward, done, _ = env.step(action)
            total_reward += reward
            steps_this_ep += 1
        eval_rewards.append(total_reward)
        eval_steps.append(steps_this_ep)
        print(f"Evaluation Episode {ep+1}: Reward = {total_reward:.2f}, Steps = {st

    print(f"\nAverage evaluation reward over {num_eval_episodes} episodes: {np.mean
    print(f"Average evaluation steps over {num_eval_episodes} episodes: {np.mean(ev


    # --- Evaluation of running a single episode with the learned policy ---
    print("\nRunning a single evaluation episode with DDQN agent (rendering)...")

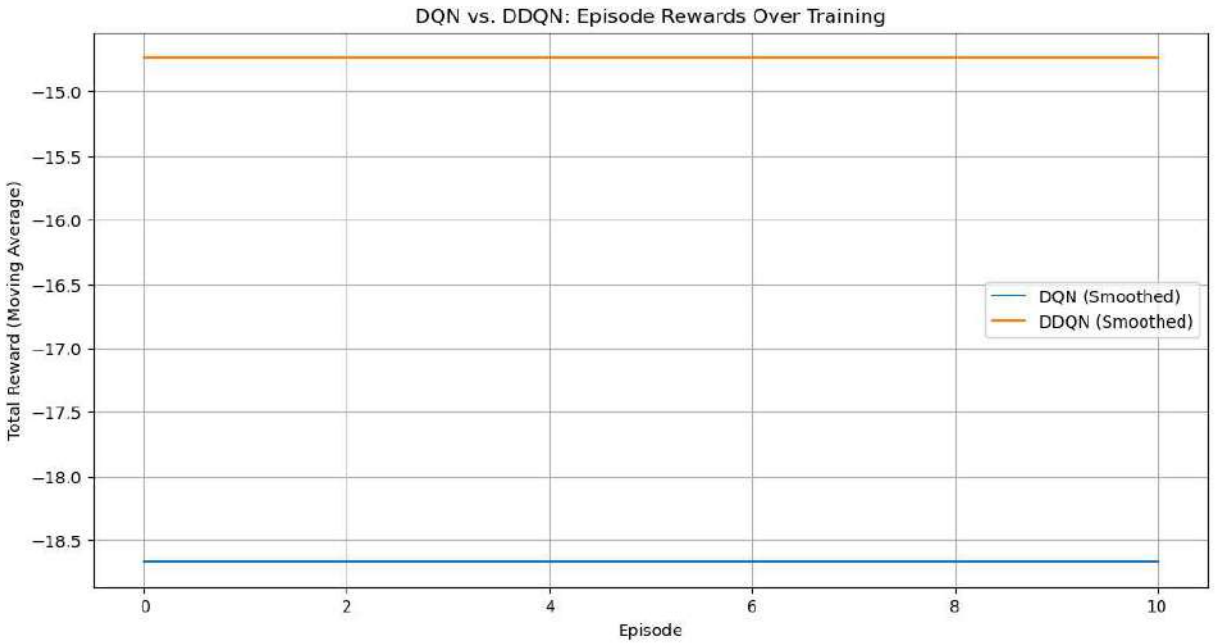
    state = env.reset()
    total_reward = 0
    steps_this_ep = 0
    done = False
    while not done and steps_this_ep < MAX_TIMESTEPS_PER_EPISODE:
        action = ddqn_agent_eval.choose_action(state) # Use learned policy
        state, reward, done, _ = env.step(action)
        total_reward += reward
        steps_this_ep += 1
        env.render() # Render the environment for this single episode
        time.sleep(0.1) # Small delay to visualize

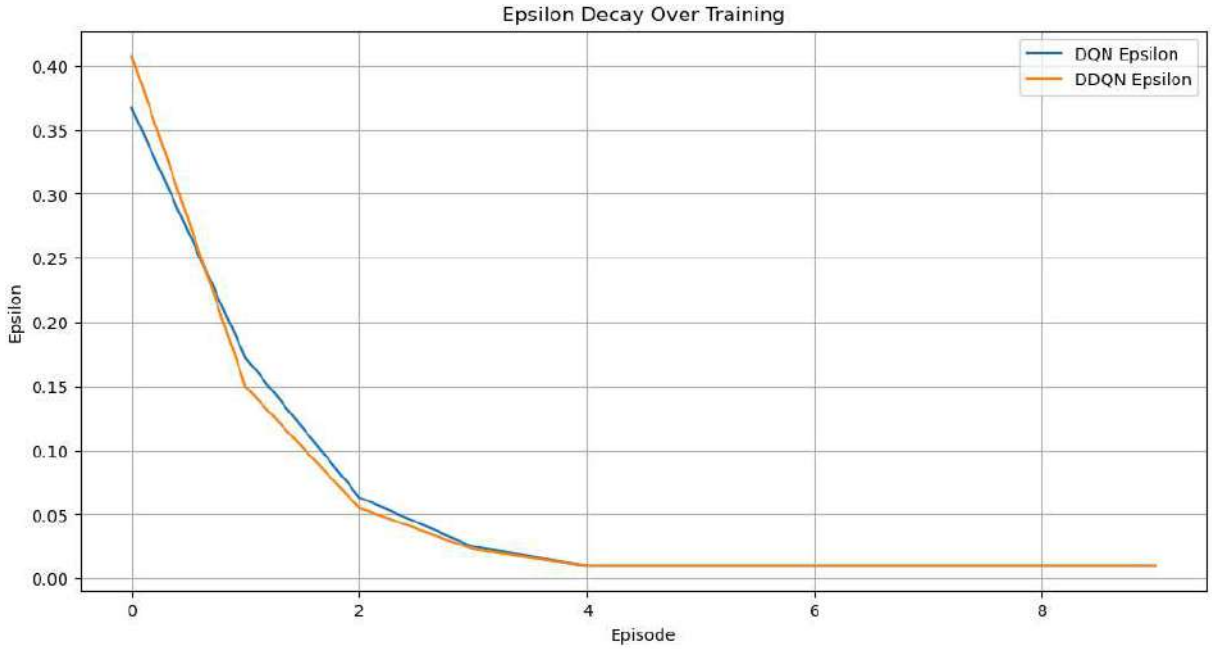
    print(f"\nSingle Evaluation Episode Complete:")
    print(f"Total Reward: {total_reward:.2f}")
    print(f"Total Steps: {steps_this_ep}")
    print(f"Final Battery Level: {env.battery_level:.1f}%")
    print(f"Final Surveillance Score: {env.total_surveillance_score:.1f}")
```

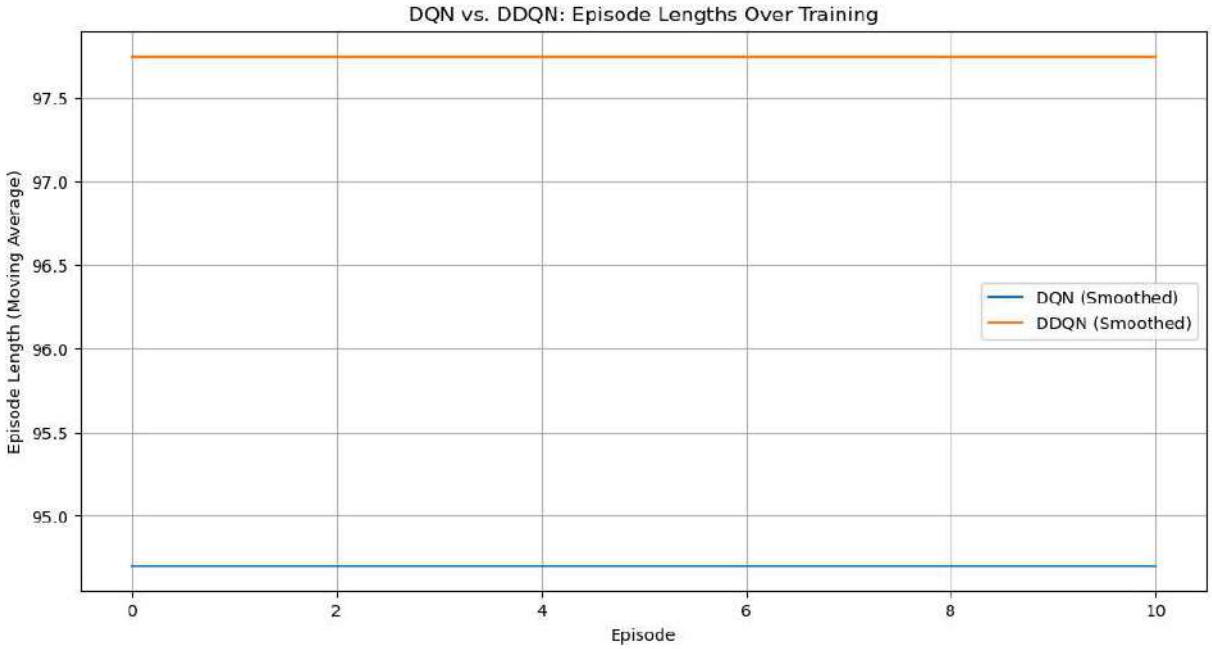
```
Training DQN Agent...
Prefilling replay buffer with 1000 random steps...
Replay buffer pre-filling complete.
[DQN] Episode 1/10 starting...
[DQN] Episode 2/10 starting...
[DQN] Episode 3/10 starting...
[DQN] Episode 4/10 starting...
[DQN] Episode 5/10 starting...
[DQN] Episode 6/10 starting...
[DQN] Episode 7/10 starting...
[DQN] Episode 8/10 starting...
[DQN] Episode 9/10 starting...
[DQN] Episode 10/10 starting...
[DQN] Episode 10/10 - Avg Reward (last 10): -37.33 - Epsilon: 0.010 - Avg Steps: 18
9.4

Training Double DQN Agent...
[DDQN] Episode 1/10 starting...
[DDQN] Episode 2/10 starting...
[DDQN] Episode 3/10 starting...
[DDQN] Episode 4/10 starting...
[DDQN] Episode 5/10 starting...
[DDQN] Episode 6/10 starting...
[DDQN] Episode 7/10 starting...
[DDQN] Episode 8/10 starting...
[DDQN] Episode 9/10 starting...
[DDQN] Episode 10/10 starting...
[DDQN] Episode 10/10 - Avg Reward (last 10): -29.46 - Epsilon: 0.010 - Avg Steps: 19
5.5

Plotting results...
```




Epsilon Decay Over Training
DQN Epsilon
DDQN Epsilon
Epsilon
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
2
4
6
8
Episode


DQN vs. DDQN: Episode Lengths Over Training
DQN (Smoothed)
DDQN (Smoothed)
Episode Length (Moving Average)
97.5
97.0
96.5
96.0
95.5
95.0
0
2
4
6
8
10
Episode

```
Analyzing learned policy (for DDQN agent)...
Running 10 evaluation episodes with DDQN agent (epsilon=0)...
Evaluation Episode 1: Reward = -20.00, Steps = 200
Evaluation Episode 2: Reward = -20.00, Steps = 200
Evaluation Episode 3: Reward = -20.00, Steps = 200
Evaluation Episode 4: Reward = -7.85, Steps = 200
Evaluation Episode 5: Reward = -20.00, Steps = 200
Evaluation Episode 6: Reward = -20.00, Steps = 200
Evaluation Episode 7: Reward = 1.24, Steps = 200
Evaluation Episode 8: Reward = 3.50, Steps = 200
Evaluation Episode 9: Reward = 24.63, Steps = 200
Evaluation Episode 10: Reward = -20.00, Steps = 200

Average evaluation reward over 10 episodes: -9.85
Average evaluation steps over 10 episodes: 200.0

Running a single evaluation episode with DDQN agent (rendering)...
----------------------
| . . . . . . . . C |
| D . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 1], Battery: 99.4%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| D . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 2], Battery: 98.9%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 3], Battery: 98.3%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 4], Battery: 97.8%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . P . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| D . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 97.2%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . P . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| D . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 96.7%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . P . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| D . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 5], Battery: 96.1%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 95.5%, Disturbance: 0.26
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 95.0%, Disturbance: 0.26
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 94.4%, Disturbance: 0.26
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 5], Battery: 93.8%, Disturbance: 0.26
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 93.3%, Disturbance: 0.26
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 92.7%, Disturbance: 0.17
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 92.2%, Disturbance: 0.17
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 5], Battery: 91.7%, Disturbance: 0.17
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 91.1%, Disturbance: 0.17
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 90.6%, Disturbance: 0.19
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 90.3%, Disturbance: 0.19
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 5], Battery: 90.1%, Disturbance: 0.19
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . P . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 89.9%, Disturbance: 0.19
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| D . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 5], Battery: 89.7%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 6], Battery: 89.1%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 88.6%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 88.3%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 88.1%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 87.9%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 87.7%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 87.5%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 87.2%, Disturbance: 0.23
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 87.0%, Disturbance: 0.22
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 86.8%, Disturbance: 0.22
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 86.6%, Disturbance: 0.22
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 86.3%, Disturbance: 0.22
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 86.1%, Disturbance: 0.22
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 85.9%, Disturbance: 0.22
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 85.7%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 85.4%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 85.2%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 85.0%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 84.8%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 84.6%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 84.3%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 84.1%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 83.9%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 83.7%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 83.4%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 83.2%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 83.0%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 82.8%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 82.5%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 82.3%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . P . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 82.1%, Disturbance: 0.23
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . D . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [1 7], Battery: 81.5%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . . D . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [2 7], Battery: 81.0%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . . . D . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [3 7], Battery: 80.4%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . . D . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [2 7], Battery: 79.8%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . . . D . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [3 7], Battery: 79.3%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . . D . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [2 7], Battery: 78.7%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . D . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [1 7], Battery: 78.1%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . . D . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [2 7], Battery: 77.6%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . D . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [1 7], Battery: 77.0%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . . D . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [2 7], Battery: 76.4%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . D . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [1 7], Battery: 75.9%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . . D . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [2 7], Battery: 75.3%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| . D . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [1 7], Battery: 74.7%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 74.2%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 73.6%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 73.0%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 72.5%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 71.9%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 71.3%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 70.8%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 70.2%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 69.6%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 69.0%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 68.5%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 67.9%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . P . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 67.3%, Disturbance: 0.27
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 66.8%, Disturbance: 0.27
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 66.6%, Disturbance: 0.27
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 66.3%, Disturbance: 0.18
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 66.1%, Disturbance: 0.18
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 65.9%, Disturbance: 0.18
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . D . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [1 7], Battery: 65.4%, Disturbance: 0.18
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . D . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [2 7], Battery: 64.8%, Disturbance: 0.18
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . D . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [3 7], Battery: 64.3%, Disturbance: 0.18
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . D . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [2 7], Battery: 63.7%, Disturbance: 0.18
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . D . . . . . . |
| P . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [3 7], Battery: 63.2%, Disturbance: 0.18
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . D . . . . . . . |
| P . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [2 7], Battery: 62.6%, Disturbance: 0.18
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . D . . . . . . |
| P . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [3 7], Battery: 62.1%, Disturbance: 0.18
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . D . . . . . . . |
| P . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [2 7], Battery: 61.5%, Disturbance: 0.18
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . D . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [1 7], Battery: 61.0%, Disturbance: 0.18
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . D . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [2 7], Battery: 60.4%, Disturbance: 0.21
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . D . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [1 7], Battery: 59.9%, Disturbance: 0.21
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 59.3%, Disturbance: 0.21
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . D . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [1 7], Battery: 58.8%, Disturbance: 0.21
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 58.2%, Disturbance: 0.29
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 58.0%, Disturbance: 0.29
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 57.7%, Disturbance: 0.29
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 57.5%, Disturbance: 0.29
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 56.9%, Disturbance: 0.29
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 56.4%, Disturbance: 0.29
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| P . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 55.8%, Disturbance: 0.29
Active POIs: 2, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 55.2%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 55.0%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 54.8%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 54.5%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 54.3%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 54.1%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 53.9%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 53.6%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 53.4%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 53.2%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 52.9%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 52.7%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 52.5%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 52.3%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 52.0%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 51.8%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 51.6%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 51.3%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 51.1%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| D . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [0 7], Battery: 50.9%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 50.7%, Disturbance: 0.29
Active POIs: 0, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| D . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [0 7], Battery: 50.4%, Disturbance: 0.29
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . D . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [1 7], Battery: 49.9%, Disturbance: 0.29
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . P |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . D . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [2 7], Battery: 49.3%, Disturbance: 0.29
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . P . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . D . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [3 7], Battery: 48.7%, Disturbance: 0.29
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . P . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . D . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [4 7], Battery: 48.1%, Disturbance: 0.29
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . P . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . D . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [5 7], Battery: 47.6%, Disturbance: 0.26
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . P . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . D . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [6 7], Battery: 47.0%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . P . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . D . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [7 7], Battery: 46.5%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . P . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . D . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [8 7], Battery: 45.9%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . P . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . D . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [8 6], Battery: 45.4%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . P . |
| . . . . . . . . . |
| . . . . . C . . D |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [8 5], Battery: 44.8%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . P . |
| . . . . . . . D . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [8 4], Battery: 44.2%, Disturbance: 0.23
Active POIs: 1, Score: 0.0
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . D . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [8 3], Battery: 43.7%, Disturbance: 0.23
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . D . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [7 3], Battery: 43.1%, Disturbance: 0.23
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . . C |
| . . . . . . . . . |
| . . . C . . . D . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [7 2], Battery: 42.6%, Disturbance: 0.23
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . . C |
| . . . . . . . D . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [7 1], Battery: 42.0%, Disturbance: 0.23
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 41.5%, Disturbance: 0.23
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 40.9%, Disturbance: 0.23
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . |
| . . . C . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . C . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| . . . . . . . . . |
| C . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 40.3%, Disturbance: 0.23
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 39.8%, Disturbance: 0.23
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 39.2%, Disturbance: 0.22
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 38.7%, Disturbance: 0.22
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 38.5%, Disturbance: 0.22
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 38.2%, Disturbance: 0.22
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 38.0%, Disturbance: 0.22
Active POIs: 2, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 37.5%, Disturbance: 0.22
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 36.9%, Disturbance: 0.22
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 36.3%, Disturbance: 0.22
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 35.8%, Disturbance: 0.22
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 35.2%, Disturbance: 0.20
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 34.7%, Disturbance: 0.20
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 34.1%, Disturbance: 0.20
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 33.6%, Disturbance: 0.20
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 33.1%, Disturbance: 0.13
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 32.5%, Disturbance: 0.13
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 32.0%, Disturbance: 0.13
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 31.5%, Disturbance: 0.13
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 30.9%, Disturbance: 0.13
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 30.4%, Disturbance: 0.13
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 29.9%, Disturbance: 0.13
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 29.3%, Disturbance: 0.13
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 28.8%, Disturbance: 0.13
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 28.3%, Disturbance: 0.13
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 27.7%, Disturbance: 0.15
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 27.2%, Disturbance: 0.15
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 26.7%, Disturbance: 0.15
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 26.1%, Disturbance: 0.15
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| P . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 25.6%, Disturbance: 0.15
Active POIs: 3, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 25.1%, Disturbance: 0.15
Active POIs: 2, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 24.5%, Disturbance: 0.15
Active POIs: 2, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . P . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 24.0%, Disturbance: 0.15
Active POIs: 2, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 23.4%, Disturbance: 0.15
Active POIs: 1, Score: 38.6
----------------------
| . . P . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 23.2%, Disturbance: 0.15
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 23.0%, Disturbance: 0.23
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 22.8%, Disturbance: 0.23
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 22.6%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 22.3%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 22.1%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 21.9%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 21.7%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 21.4%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 21.2%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 21.0%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 20.8%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 20.5%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . . . . . C |
----------------------
Drone Pos: [7 0], Battery: 20.3%, Disturbance: 0.25
Active POIs: 0, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . P . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 20.1%, Disturbance: 0.25
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . P . . . C |
----------------------
Drone Pos: [7 0], Battery: 19.8%, Disturbance: 0.25
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . P . . . C |
----------------------
Drone Pos: [7 0], Battery: 19.6%, Disturbance: 0.25
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . P . . . C |
----------------------
Drone Pos: [7 0], Battery: 19.4%, Disturbance: 0.25
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . P . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 19.2%, Disturbance: 0.25
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . P . . . C |
----------------------
Drone Pos: [7 0], Battery: 18.9%, Disturbance: 0.25
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . P . . . C |
----------------------
Drone Pos: [7 0], Battery: 18.7%, Disturbance: 0.25
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . P . . . C |
----------------------
Drone Pos: [7 0], Battery: 18.5%, Disturbance: 0.25
Active POIs: 1, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . P . . . |
| . . . C . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . C . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| . . . . . . . . . . |
| C . . . . P . . . C |
```

```
----------------------
Drone Pos: [7 0], Battery: 18.3%, Disturbance: 0.25
Active POIs: 2, Score: 38.6
----------------------
| . . . . . . . D . C |
| . . . . . . . P . . . |
| . . . C . . . . . . . |
| . . . . . . . . . . . |
| . . . . . . . . . . . |
| . . . . . C . . . . . |
| . . . . . . . . . . . |
| . . . . . . . . . . . |
| . . . . . . . . . . . |
| C . . . . P . . . C |
----------------------
Drone Pos: [7 0], Battery: 17.7%, Disturbance: 0.25
Active POIs: 2, Score: 38.6

Single Evaluation Episode Complete:
Total Reward: 18.64
Total Steps: 200
Final Battery Level: 17.7%
Final Surveillance Score: 38.6
```

In [30]:

```python
# ----------------------------
# Section: Visualize Agent Movement + Battery + Disturbance
# ----------------------------
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import numpy as np

def visualize_episode(agent: DQNAgent, env: DroneSurveillanceEnv, steps: int = MAX_
    """
    Runs a single episode with the trained agent (using its learned policy without
    and generates plots to visualize:
      - The agent's path across the grid.
      - The drone's battery level over time.
      - The atmospheric disturbance level over time. [1]

    Args:
        agent (DQNAgent): The trained agent instance (DQN or DDQN).
        env (DroneSurveillanceEnv): The environment instance.
        steps (int): The maximum number of steps to run for visualization. Defaults
    """
    state = env.reset() # Reset the environment for a fresh episode
    trajectory = [] # List to store the drone's position at each timestep
    battery_history = [] # List to store battery level at each timestep
    disturbance_history = [] # List to store disturbance level at each timestep

    # Run the episode for visualization
    for step in range(steps):
        # Agent chooses the greedy action (action with the highest predicted Q-valu
        # np.newaxis adds a batch dimension for the model prediction
        action = np.argmax(agent.model.predict(state[np.newaxis], verbose=0)[0])
        # Take a step in the environment with the chosen action
        next_state, reward, done, _ = env.step(action)
```

```
        # Record relevant state information if available in the environment
        if hasattr(env, "drone_pos"):   # Check if env has 'drone_pos' attribute
            trajectory.append(env.drone_pos.copy()) # Append a copy of the drone's

        if hasattr(env, "battery_level"): # Check if env has 'battery_level' attrib
            battery_history.append(env.battery_level) # Append the battery level

        if hasattr(env, "atmospheric_disturbance"): # Check if env has 'atmospheric
            disturbance_history.append(env.atmospheric_disturbance) # Append the di

        state = next_state # Update the current state
        if done:
            break # End the visualization episode if the environment signals 'done'

    # ---- Create figure with 3 subplots stacked vertically ----
    fig, axs = plt.subplots(3, 1, figsize=(8, 14)) # 3 rows, 1 column

    # --- 1. Plot the path on the grid --- [10]
    grid_size = env.grid_size  # Get grid dimensions from the environment
    grid = np.zeros((grid_size[1], grid_size[0])) # Initialize a grid array (note:

    # Mark Charging Stations on the grid visualization
    if hasattr(env, "charging_stations"):
        for (x, y) in env.charging_stations:
            # Ensure position is within bounds and mark with a specific value (2)
            if 0 <= x < grid_size[0] and 0 <= y < grid_size[1]:
                 grid[y, x] = 2   # 2 = charging station marker value

    # Mark Active POIs on the grid visualization
    # Note: This will only show POIs that were active at the END of the visualizati
    # A more complex visualization could show POI spawning/disappearing over time.
    if hasattr(env, "active_pois"):
        for poi in env.active_pois:
            x, y = poi['pos']
            # Ensure position is within bounds and mark with a specific value (3)
            if 0 <= x < grid_size[0] and 0 <= y < grid_size[1]:
                grid[y, x] = 3   # 3 = point of interest marker value

    # Define a colormap for visualization: White (empty), Black (not used), Blue (C
    cmap = mcolors.ListedColormap(["white", "black", "blue", "green"])
    # Display the grid, using 'origin="lower"' to match typical grid coordinates (0
    axs[0].imshow(grid, cmap=cmap, origin="lower")

    # Plot the drone's trajectory
    if trajectory:
        xs, ys = zip(*trajectory) # Separate X and Y coordinates from the trajector
        axs[0].plot(xs, ys, marker="o", color="red", label="Agent Path") # Plot the
        # Mark the starting position with a star
        axs[0].scatter(xs[0], ys[0], c="gold", marker="*", s=200, label="Start")

    # Set title and add legend for the grid plot
    axs[0].set_title("Agent Movement Across Grid")
    axs[0].legend()

    # --- 2. Plot Battery level over time ---
```

```
    if battery_history:
        axs[1].plot(battery_history, color="blue", linewidth=2) # Plot battery hist
        axs[1].set_ylim(0, 100) # Set Y-axis limits for battery percentage
        axs[1].set_ylabel("Battery (%)") # Label Y-axis
        axs[1].set_xlabel("Time step") # Label X-axis
        axs[1].set_title("Battery Level Over Time") # Set plot title
        axs[1].grid(True) # Add grid

    # --- 3. Plot Atmospheric Disturbance over time ---
    if disturbance_history:
        axs[2].plot(disturbance_history, color="orange", linewidth=2) # Plot distur
        axs[2].set_ylim(0, 1) # Set Y-axis limits for disturbance level [0, 1]
        axs[2].set_ylabel("Disturbance Level") # Label Y-axis
        axs[2].set_xlabel("Time step") # Label X-axis
        axs[2].set_title("Atmospheric Disturbance Over Time") # Set plot title
        axs[2].grid(True) # Add grid

    # Adjust layout to prevent overlapping titles/labels
    plt.tight_layout()
    # Display the plots
    plt.show()


# Example usage after training:
# Make sure to pass the trained agent and the environment instance
# visualize_episode(dqn_agent, env) # Example for DQN agent
# visualize_episode(ddqn_agent, env) # Example for DDQN agent
```

Hyperparameter Tuning & Discussion: (1 Mark)